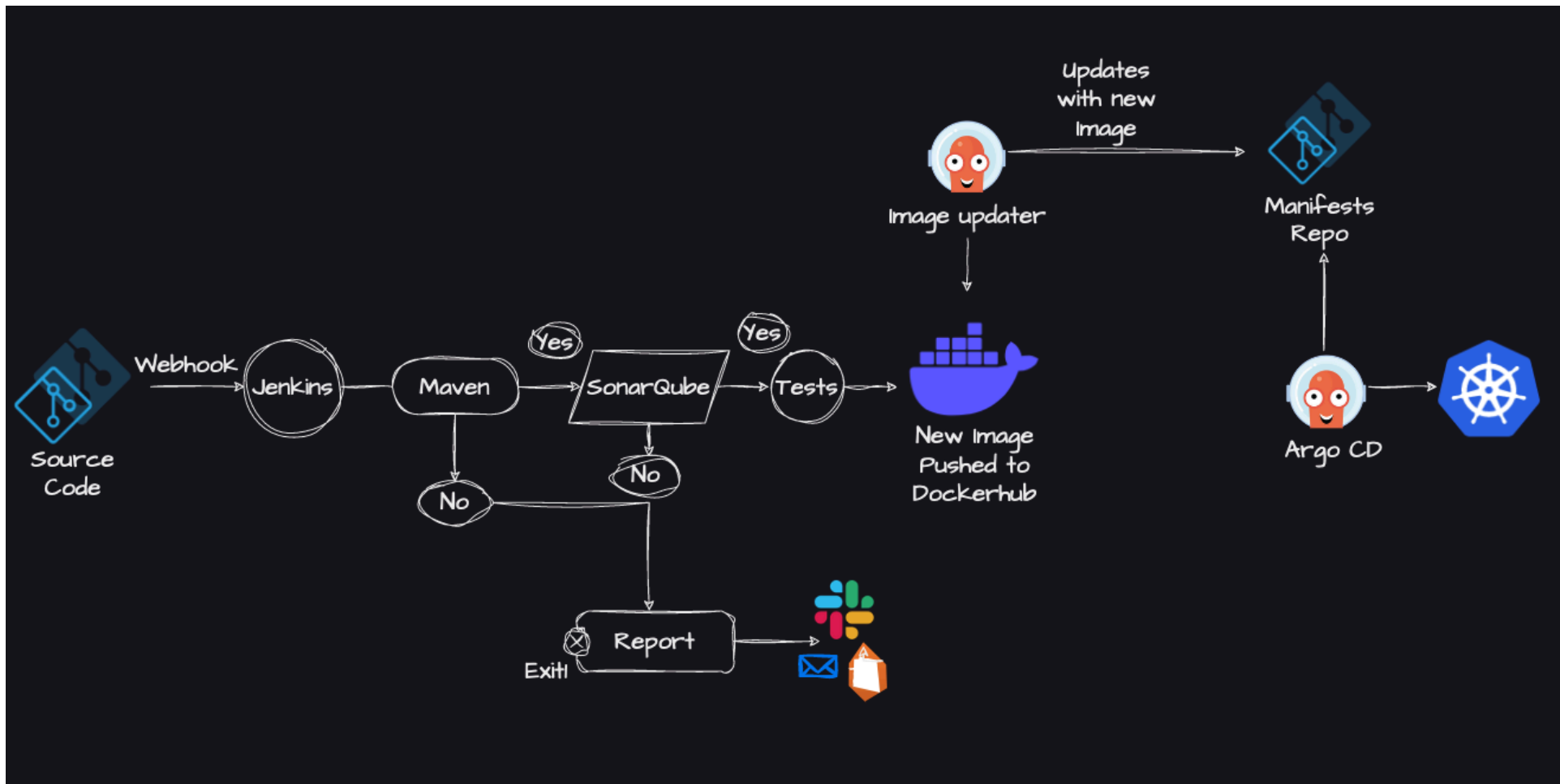


Jenkins Pipeline for Java based application using Maven, SonarQube, Argo CD, Helm and Kubernetes

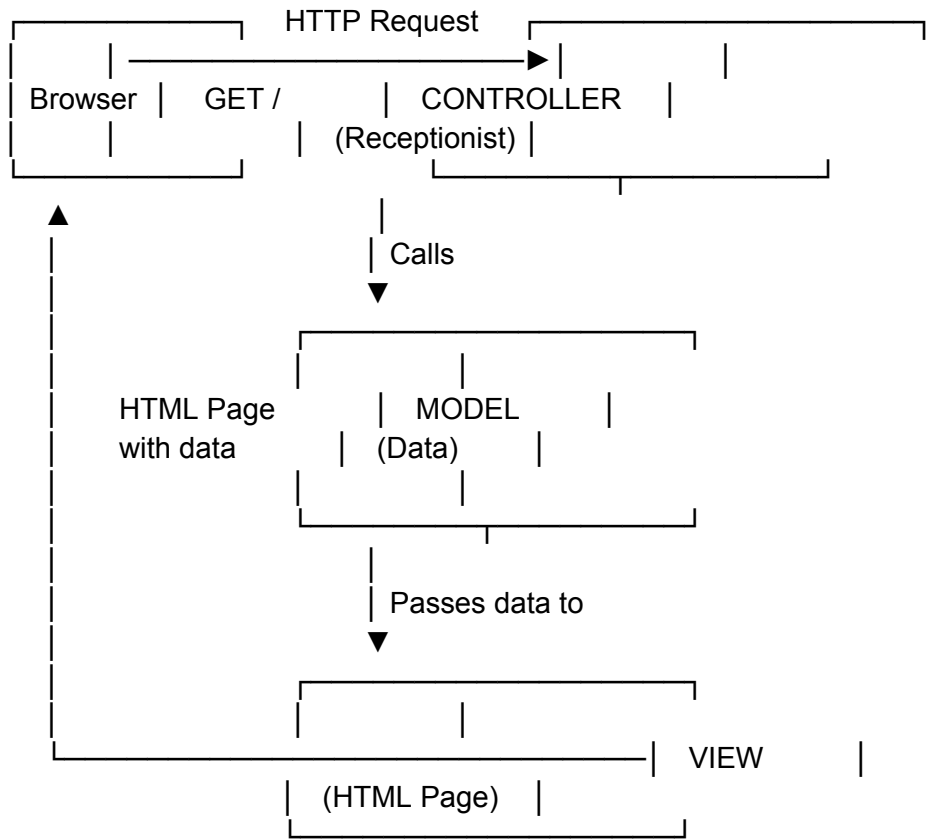


Here are the step-by-step details to set up an end-to-end Jenkins pipeline for a Java application using SonarQube, Argo CD, Helm, and Kubernetes:

About Application: Spring Boot based Java web application (Spring Boot is a tool that helps you build Java web applications quickly without writing lots of configuration code.)

This is a simple Spring Boot-based Java application that can be built using Maven. Spring Boot is a tool that helps you build Java web applications quickly without writing lots of configuration code. Spring Boot dependencies are managed via the pom.xml file at the repository's root.

This is an MVC (Model-View-Controller) architecture based application where controller returns a page with title and message attributes to the view.



#This starts: Embedded Tomcat Server (built into Spring Boot) Spring Boot comes with embedded Tomcat (or Jetty/Undertow) built right into the JAR file. This is one of Spring Boot's biggest advantages!
Your application code listens on port 8080

Pom.xml is the dependencies for Java Maven project

Building the archive is done by Maven by referring to the pom.xml file

Step 1) Execute the application locally and access it using your browser – To undersatnd the app

1.1) Checkout the repo and move to the directory

```
$ git clone https://github.com/iam-veeramalla/Jenkins-Zero-To-Hero/java-maven-sonar-argocd-helm-k8s/sprint-boot-app
$ cd java-maven-sonar-argocd-helm-k8s/sprint-boot-app
```

1.2) Execute the Maven targets to generate the artifacts

```
$ sudo apt install maven
$ mvn clean package
```

The above maven target stroes the artifacts to the target directory. You can either execute the artifact on your local machine (or) run it as a Docker container.

**** Note:** To avoid issues with local setup, Java versions and other dependencies, I would recommend the docker way. ******

1.3) Execute locally (Java 11 needed) and access the application on http://localhost:8080

```
$ java -jar target/spring-boot-web.jar
```

OR)

The Docker way : Build the Docker Image

```
$ docker build -t ultimate-cicd-pipeline:v1 .
# docker run -d -p 8010:8080 -t ultimate-cicd-pipeline:v1
```

Hurray !! Access the application on http://<ip-address>:8010

Step 2: Deploy in AWS Cloud:

2.1) Create AWS instance T2.large. (We need to install docker, and SonarServer as well)

2.2) Install Jenkins

Pre-Requisites:

- Java (JDK)
- Run the below commands to install Java and Jenkins

Pre-Requisites:

```
sudo apt update
sudo apt install fontconfig openjdk-21-jre java -version
```

2.2 Jenkins Installation:

```
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc
https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]"
https://pkg.jenkins.io/debian-stable binary/ | sudo tee
/etc/apt/sources.list.d/jenkins.list > /dev/null
sudo apt update
sudo apt install jenkins
```

<https://www.jenkins.io/doc/book/installing/linux/#debianubuntu>
...

Jenkins Run on Port 8080: (<http://3.80.139.103:8080/>)

3) Create a Pipeline:

- 1) Select Create a new Job.
- 2) In Pipeline Option >> Select Pipeline Script from SCM, select “GitHub”
- 3) Give the Repo URL: <https://github.com/Ruzzel6/Jenkins-Zero-To-Hero.git>
- 4) Provide the Path for Jenkinsfile: java-maven-sonar-argocd-helm-k8s/sprint-boot-app/JenkinsFile
- 5) So you can keep the JenkinsFile in the same repo as the code. If needed.
- 6) Jenkins Pipeline has all the code for continuous integration.

4) Using a Docker container as an Agent for running the jobs/stages in the Pipeline:

Which is more cost-effective and has less configuration, instead of making a Master/Slave configuration. Each stage would be run in a container. Once all the stages are completed in the pipeline, the Docker container is killed.

4.1) Two Things: You need to select the correct Image in the Jenkins file and install the Docker plugin in the Jenkins Server:

4.1.1) Install the Docker Pipeline plugin in Jenkins:

Log in to Jenkins.
Go to Manage Jenkins > Manage Plugins.
In the Available tab, search for "Docker Pipeline".
Select the plugin and click the Install button.

4.2) Install SonarPlugin: SonarQube Scanner Version 2.18.3
Restart Jenkins after the plugin is installed.

5) Install SonarQube on the server:

Configure a Sonar Server locally
System Requirements: Java 17+ (Oracle JDK, OpenJDK, or AdoptOpenJDK)

```
$ sudo apt update && sudo apt install unzip -y

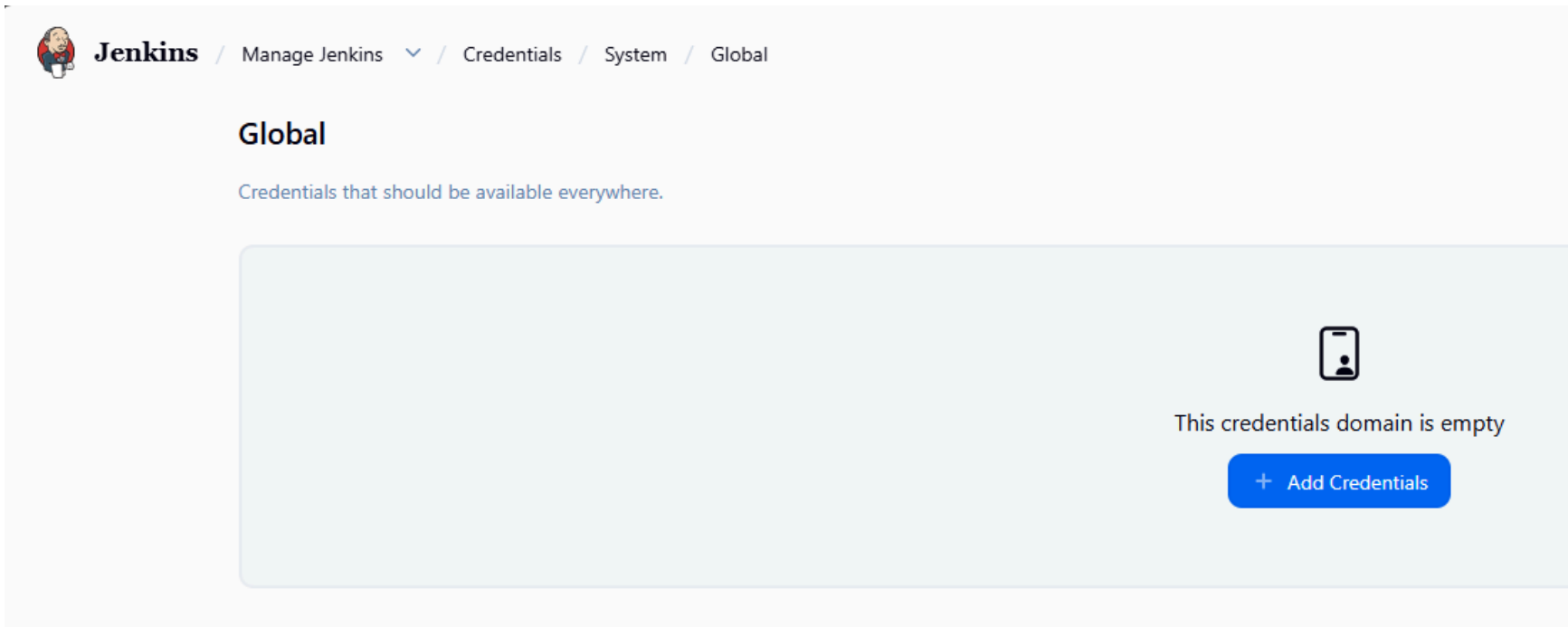
-adduser sonarqube
-//Switch to user sonarqube and download the zipfile below

wget https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-10.4.1.88267.zip
$ unzip *
sonarqube@ip-172-31-22-14:~$ chown -R sonarqube:sonarqube /home/sonarqube/sonarqube-10.4.1.88267
$ chmod -R 755 /home/sonarqube/sonarqube-10.4.1.88267
$ cd /home/sonarqube/sonarqube-10.4.1.88267/bin/linux-x86-64
$ ./sonar.sh start
/usr/bin/java
Starting SonarQube...
Started SonarQube.
```

Runs on Port 9000

6) Authenticate Sonar with Jenkins

6.1) Go TO My account >> Security :::: “Generate Token” (Create a User Token)
Eg: squ_a443756b97b7a4e33b2accf21542fcfb30ad488f
6.2) Copy this in Jenkins:
Go to Jenkins >> Manage Jenkins >> Under ‘security’ >> “Credentials” >> Go To “System” >> “Global”
You will have to add credentials here:



6.3) Type or Kind: Secret text:

<

Add Secret text

×

Scope ?

Global (Jenkins, nodes, items, all child items, etc) ▾

Secret

.....

ID ?

sonarqube

7) Docker Slave Configuration

Install Docker: `sudo apt install docker.io`

7.2) Grant Jenkins user and Ubuntu user permission to the Docker daemon.

```
sudo su -
usermod -aG docker jenkins
usermod -aG docker ubuntu
systemctl restart docker
```

Restart Jenkins either `url/restart` or `systemctl restart`.

8: Create Kubernetes cluster:

Create a Kubernetes cluster:

Kubernetes/kubernetes_install/kubeadm at main · Ruzzel6/Kubernetes

Kubeadm Installation Guide

Prerequisites

Ubuntu OS (Xenial or later), sudo privileges Internet access t2.medium instance type or higher

AWS Setup

Ensure that all instances are in the same Security Group. Expose port 6443 in the Security Group to allow worker nodes to join the cluster. Expose port 22 in the Security Group to allow SSH access to manage the instance.

Execute on Both "Master" & "Worker" Nodes

Disable Swap: Required for Kubernetes to function correctly.

```
sudo swapoff -a
```

Load Necessary Kernel Modules: Required for Kubernetes networking.

Explicitly loads kernel modules (overlay and br_netfilter) and sets sysctl parameters for networking.

Ensures proper networking for Kubernetes.

Includes commands to verify the loaded modules.

```
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
```

```
sudo modprobe overlay
```

```
sudo modprobe br_netfilter
```

Set Sysctl Parameters: Helps with networking.

```
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
```

```
net.bridge.bridge-nf-call-iptables = 1
```

```
net.bridge.bridge-nf-call-ip6tables = 1
```

```
net.ipv4.ip_forward = 1
```

EOF

```
sudo systemctl --system
lsmod | grep br_netfilter
lsmod | grep overlay
Install Containerd:
sudo apt-get update
sudo apt-get install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu $(.
/etc/os-release && echo \"${VERSION_CODENAME}\") stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt-get update
sudo apt-get install -y containerd.io

containerd config default | sed -e 's/SystemdCgroup = false/SystemdCgroup = true/' -e 's/sandbox_image =
"registry.k8s.io/pause:3.6"/sandbox_image = "registry.k8s.io/pause:3.9"/' | sudo tee /etc/containerd/config.toml

sudo systemctl restart containerd
sudo systemctl status containerd
Install Kubernetes Components:
sudo apt-get update
sudo apt-get install -y apt-transport-https ca-certificates curl gpg

curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.29/deb/Release.key | sudo gpg --dearmor -o
/etc/apt/keyrings/kubernetes-apt-keyring.gpg

echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.29/deb/ ' | sudo tee
/etc/apt/sources.list.d/kubernetes.list

sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
Execute ONLY on the "Master" Node
Initialize the Cluster:
sudo kubeadm init
Set Up Local kubeconfig:
mkdir -p "$HOME"/.kube
sudo cp -i /etc/kubernetes/admin.conf "$HOME"/.kube/config
sudo chown "$(id -u)": "$(id -g)" "$HOME"/.kube/config
Install a Network Plugin (Calico):
calico/weave/Flannel
kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.26.0/manifests/calico.yaml
Generate Join Command:
kubeadm token create --print-join-command
Copy this generated token for next command.

Execute on ALL of your Worker Nodes
Perform pre-flight checks:
sudo kubeadm reset pre-flight checks
Paste the join command you got from the master node and append --v=5 at the end:

sudo kubeadm join <private-ip-of-control-plane>:6443 --token <token> --discovery-token-ca-cert-hash sha256:<hash> --cri-socket
"unix:///run/containerd/containerd.sock" --v=5
Verify Cluster Connection
kubectl get nodes
```

9) Install ArgoCD using operator:

Argo CD is a GitOps continuous delivery (CD) tool for Kubernetes.

Its job is to keep your Kubernetes cluster synchronized with the manifests stored in a Git repository.

You Can:

Store your Kubernetes YAML files in Git.

Tell Argo CD where the Git repository is.

Argo CD continuously watches the repository.

Whenever Git changes, Argo CD automatically applies those changes to the cluster.

Kubernetes Operator:

An Operator is a Kubernetes application that automates the management of another application. It extends Kubernetes with application-specific knowledge.

the Operator creates and maintains the Argo CD components automatically.

What does the Argo CD Operator do?

It automates tasks such as:

Installing Argo CD

Upgrading Argo CD

Configuring Argo CD

Restarting components if needed

Managing certificates and other configuration (depending on the Operator's features)

Without the Operator, you'd manage these yourself.

9) Install ArgoCD using operator:

Install k8s controllers using operators: :: Go To:: [OperatorHub.io | The registry for Kubernetes Operators](https://operatorhub.io)

>>>operatorhub.io

Search ArgoCD and click on install to get the steps:



Argo CD

0.18.0 provided by Argo CD Community

Install on Kubernetes

1. Install Operator Lifecycle Manager (OLM), a tool to help manage the Operators running on your cluster.

```
$ curl -sL https://github.com/operator-framework/operator-lifecycle-manager/releases/download/v0.45.0/install.sh | bash -s v0.45.0
```

2. Install the operator by running the following command:

[What happens when I execute this command?](#)

```
$ kubectl create -f https://operatorhub.io/install/argocd-operator.yaml
```

This Operator will be installed in the "operators" namespace and will be usable from all namespaces in the cluster.

3. After install, watch your operator come up using next command.

```
$ kubectl get csv -n operators
```

To use it, checkout the custom resource definitions (CRDs) introduced by this operator to start using it.

Step 10: Put Docker credentials and GitHub credentials inside Jenkins Manage :

DockerHub credentials: 'Username and Password'

Go to manage Jenkins >> Credentials >> System >> Global >> Add Credentials :: Like we did before

User : ruzzel06(dockerhubuser/pass)

Select Username and Password:

<

Add Username with password

×

Scope

?

Global (Jenkins, nodes, items, all child items, etc)

▼

Username

ruzzel06

☐

 Treat username as secret

?

Password

.....

👁

ID

?

docker-cred

Description

?

Create

GitHub Cred:
Use Secret Text and add secret token from GitHub >> Developer >> Auth >> Generate token

Step 11:
=====

You would need to build a Jenkins agent(Jenkins build agent) and push it to your Docker Hub:

DockerFile
FROM maven:3.9.11-eclipse-temurin-21

```
RUN apt-get update && \
  apt-get install -y \
    docker.io \
    git \
    curl \
    unzip && \
  rm -rf /var/lib/apt/lists/*
```

```
build -t ruzzel06/maven-docker-agent:v1 .
docker login
docker push ruzzel06/jenkins-project-v:v1
```

=====

Step 12: Next, to the CD Part, create ArgoCD controller:

Install ArgoCD operator in Kubernetes cluster: <https://operatorhub.io/operator/argocd-operator>

[OperatorHub.io | The registry for Kubernetes Operators](https://operatorhub.io/operator/argocd-operator)

12) Create ArgoCD controller:

```
apiVersion: argoproj.io/v1alpha1
kind: ArgoCD
metadata:
  name: example-argocd
labels:
```

```
example: basic
spec:
  server:
    service:
      type: NodePort
```

Apply this argocd-basic.yaml

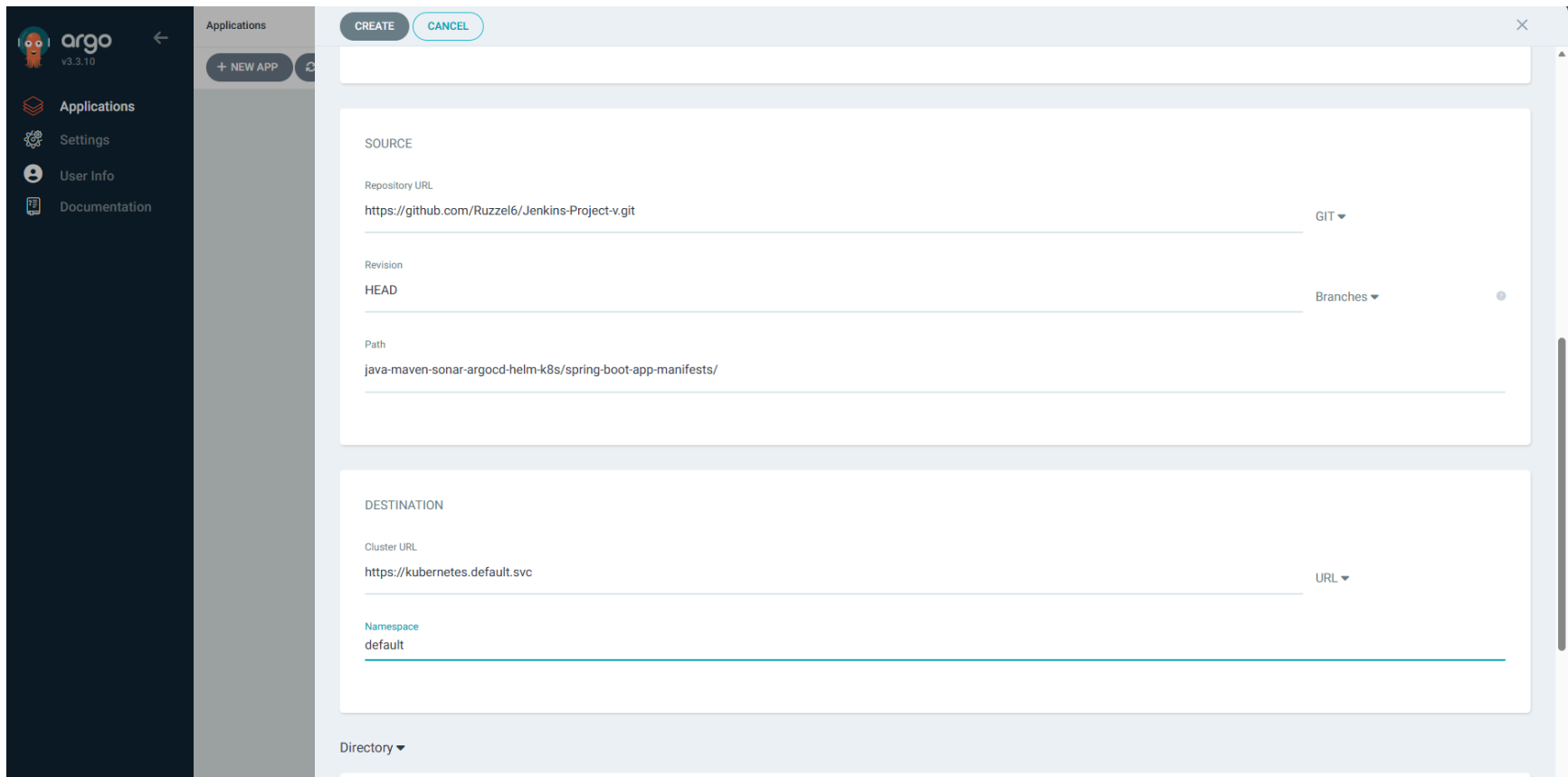
```
kubectl get pods -w
NAME                                READY STATUS      RESTARTS   AGE
argocd-application-controller-0    0/1   ContainerCreating 0        36s
argocd-redis-856746798f-bhv25      0/1   ContainerCreating 0        36s
argocd-repo-server-78569cc65-wzbxn 0/1   Init:0/1         0        36s
argocd-server-f7d8f7765-2vgtr      0/1   ContainerCreating 0        36s
webserver-85c5d948d9-ks6zq         1/1   Running          20 (45h ago) 139d
$ kubectl get svc
NAME                                TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)          AGE
argocd-metrics                      ClusterIP    10.96.62.61   <none>       8082/TCP         52s
argocd-redis                        ClusterIP    10.96.230.46  <none>       6379/TCP         52s
argocd-repo-server                  ClusterIP    10.96.168.208 <none>       8081/TCP,8084/TCP 52s
argocd-server                       NodePort     10.96.216.4   <none>       80:32032/TCP,443:31681/TCP 52s
example-argocd-server-metrics       ClusterIP    10.96.133.23  <none>       8083/TCP         52s
kubernetes                          ClusterIP    10.96.0.1     <none>       443/TCP          139d
ruzzel@Ruzzel:~/ArgoCD$
```

Once this is done, ArgoCD is accessible to get the admin user and pass

```
$ kubectl get secrets
$ kubectl edit secrets example-argocd-cluster
```

```
$ echo b0IyZ0NOMFp0aWFWdTUzT1FQc1RlcmZFdnc4QXIJbFg= | base64 -d
oB2gCN0ZtiaVu53OQPstHzfEvw8Ayll
```

Configure ArgoCD:



- Stage 1: Checkout the source code from Git.
- Stage 2: Build the Java application using Maven.
- Stage 3: Run unit tests using JUnit and Mockito.
- Stage 4: Run SonarQube analysis to check the code quality.
- Stage 5: Package the application into a JAR file.

Stage 6: Deploy the application to a test environment using Helm.
Stage 7: Run user acceptance tests on the deployed application.
Stage 8: Promote the application to a production environment using Argo CD.

```
pipeline {
  agent {
    docker {
      image 'ruzzel06/maven-docker-agent:v1'
      args '--user root -v /var/run/docker.sock:/var/run/docker.sock' // mount Docker socket to access the host's Docker daemon
    }
  }
  stages {
    stage('Checkout') {
      steps {
        sh 'echo passed'
        //checkout scm
        //git branch: 'main', url: 'https://github.com/Ruzzel6/Jenkins-Project-v.git'
      }
    }
    stage('Build and Test') {
      steps {
        sh 'ls -ltr'
        // build the project and create a JAR file
        sh 'cd java-maven-sonar-argocd-helm-k8s/spring-boot-app && mvn clean package'
      }
    }
    stage('Static Code Analysis') {
      environment {
        SONAR_URL = "http://13.221.20.39:9000"
      }
      steps {
        withCredentials([string(credentialsId: 'sonarqube', variable: 'SONAR_AUTH_TOKEN')]) {
          sh 'cd java-maven-sonar-argocd-helm-k8s/spring-boot-app && mvn sonar:sonar -Dsonar.login=$SONAR_AUTH_TOKEN -Dsonar.host.url=${SONAR_URL}'
        }
      }
    }
    stage('Build and Push Docker Image') {
      environment {
        DOCKER_IMAGE = "ruzzel06/jenkins-project-v:${BUILD_NUMBER}"
        // DOCKERFILE_LOCATION = "java-maven-sonar-argocd-helm-k8s/spring-boot-app/Dockerfile"
        REGISTRY_CREDENTIALS = credentials('docker-cred')
      }
      steps {
        script {
          sh 'cd java-maven-sonar-argocd-helm-k8s/spring-boot-app && docker build -t ${DOCKER_IMAGE} .'
          def dockerImage = docker.image("${DOCKER_IMAGE}")
          docker.withRegistry('https://index.docker.io/v1/', "docker-cred") {
            dockerImage.push()
          }
        }
      }
    }
    stage('Debug') {
      steps {
        sh ""
        echo "Current Directory:"
        pwd

        echo
        echo "Workspace:"
        echo $WORKSPACE

        echo
        echo "Files:"
        ls -la

        echo
        echo "Git:"
        git rev-parse --show-toplevel || true

        echo
      }
    }
  }
}
```

```
    echo "Git status:"
    git status || true
    ""
  }
}
stage('Update Deployment File') {
  environment {
    GIT_REPO_NAME = "Jenkins-Project-v"
    GIT_USER_NAME = "Ruzzel6"
  }
  steps {
    withCredentials([string(credentialsId: 'github', variable: 'GITHUB_TOKEN')]) {
      sh ""
      git config --global --add safe.directory ${WORKSPACE}
      git config user.email "ruzzel6@gmail.com"
      git config user.name "ruzzel6"
      git remote set-url origin https://${GITHUB_TOKEN}@github.com/${GIT_USER_NAME}/${GIT_REPO_NAME}.git
      git checkout main
      git pull origin main

      sed -i "s#ruzzel06/jenkins-project-v:.*/ruzzel06/jenkins-project-v:${BUILD_NUMBER}#g" \
      java-maven-sonar-argocd-helm-k8s/spring-boot-app-manifests/deployment.yml

      git add java-maven-sonar-argocd-helm-k8s/spring-boot-app-manifests/deployment.yml

      git diff --cached --quiet || \
      git commit -m "Update deployment image to version ${BUILD_NUMBER}"

      git push origin main
    }
  }
}
}
```